

ED-POMDP

DE LA CROYANCE À L'ACTION

Estimation d'état, recherche opérationnelle et apprentissage de politiques

Document autonome de niveau avancé. Il reproduit sans modification scientifique l'ancienne Annexe G du guide ED-POMDP en clair v1.8. Les deux pages liminaires ajoutent seulement le contrat de lecture, les prérequis et des repères terminologiques.

Prérequis conseillés

Probabilités conditionnelles, espérance, mise à jour bayésienne, optimisation, notions de machine learning et lecture générale d'un MDP/POMDP.

Repères terminologiques du companion

Les familles d'estimateurs citées sont un panorama de choix possibles, pas un cours complet. Le texte conserve certains noms anglais lorsqu'ils correspondent aux termes cano- niques de la littérature.

MAP

Maximum a posteriori : état le plus probable après observation.

Ensemble conforme

Ensemble de prédictions muni d'une garantie de couverture sous hypothèses données.

Marginalisation

Sommer ou intégrer les autres variables pour ne garder que la distribution d'intérêt.

Bandit contextuel

Choix répété d'une action à partir d'un contexte, sans modèle d'état complet.

HMM

Modèle de Markov caché : état latent évolutif ob- servé indirectement.

RL model-based

Apprentissage ou planification avec un modèle explicite des transitions.

Filtre de Kalman

Estimateur récursif pour un modèle linéaire gaus- sien.

RL model-free

Apprentissage direct d'une valeur ou politique sans modèle explicite complet.

Filtre particulaire / SMC

Approximation d'une distribution par un ensemble de particules pondérées.

Offline RL

Apprentissage d'une politique à partir de données historiques, sans exploration en ligne.

Inférence amortie

Modèle appris qui produit rapidement une approxi- mation de la croyance.

Contrainte dure

Condition qui ne peut pas être compensée par un gain sur un autre objectif.

Annexe G — De l'observation à la croyance, puis de la croyance à l'action

La question naïve qui ouvre cette annexe

Une croyance b doit-elle être renseignée manuellement ? Ne pourrait-on pas estimer automatiquement $(\hat{s}, \hat{e})$ à partir des tests, traces et incidents, éventuellement avec du machine learning, puis utiliser la recherche opérationnelle ou l'apprentissage par renforcement pour choisir la prochaine action ?

La réponse est **oui, mais pas en confondant les objets**. Une distribution de croyance, un meilleur état estimé, une représentation latente apprise et une politique de décision ont des rôles différents. Les réunir dans une même chaîne est possible ; les remplacer les uns par les autres sans précaution détruit précisément l'incertitude que le POMDP doit gouverner.

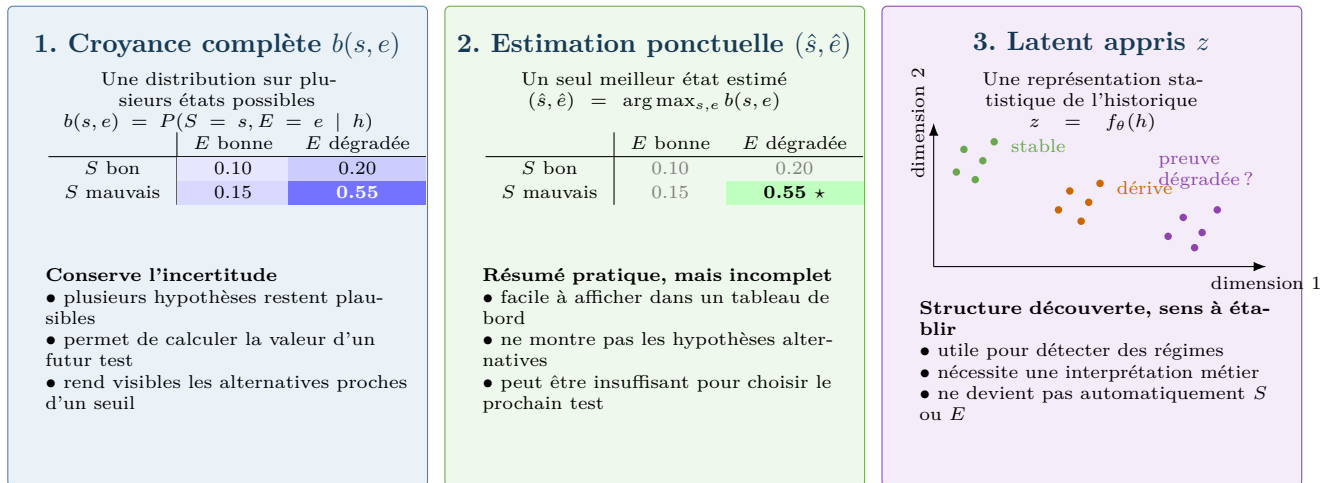
G.1 Les quatre objets à ne pas confondre

Objet	Écriture	Question à laquelle il répond	Ce qu'il perd ou exige
Croyance complète	$b_t(s, e) = P(S_t = s, E_t = e \mid h_t)$	Quels états restent possibles, avec quelles probabilités ?	Exige un modèle ou un estimateur probabiliste.
Estimation ponctuelle	$(\hat{s}_t, \hat{e}_t)$	Quel état paraît actuellement le plus plausible ?	Résume mais masque les alternatives et l'incertitude.
Représentation latente apprise	$z_t = f_\theta(h_t)$	Quels régimes ou structures statistiques le modèle détecte-t-il ?	Le sens métier de z n'est pas garanti.
Politique de décision	$\pi(b_t) = a_t$ ou $Q(b_t, a)$	Quelle action faut-il prendre compte tenu de la croyance et des conséquences ?	Exige des coûts, contraintes, seuils et une autorité.

Phrase de séparation

L'estimateur répond à « que croyons-nous ? ». L'optimiseur répond à « que devons-nous faire avec cette croyance ? ». Un modèle latent répond seulement à « quelle structure statistique ai-je découverte ? ». Ces réponses peuvent être enchaînées, mais elles ne sont pas synonymes.

G.2 Représentation visuelle : distribution, point estimé et latent appris



Même estimation, incertitudes différentes. Considérons deux croyances sur les quatre états joints :

$$b_A = (0.01, 0.04, 0.05, 0.90), \quad b_B = (0.23, 0.22, 0.24, 0.31).$$

Dans les deux cas, le MAP est « système mauvais, preuve dégradée ». Pourtant, l'entropie vaut environ 0.61 bit pour b_A et 1.99 bits pour b_B . La première croyance est presque certaine ; la seconde reste très diffuse. Le même $(\hat{s}, \hat{e})$ ne justifie donc pas nécessairement la même prochaine action.

G.3 Une croyance peut-elle être calculée automatiquement ?

Oui. C'est précisément le rôle d'un **estimateur d'état**. Il ne « voit » pas directement S_t et E_t ; il combine l'historique des observations, le modèle d'évolution et le modèle de mesure.

La mise à jour peut être décomposée en deux temps.

1. Prédire l'état avant la nouvelle observation :

$$\tilde{b}_t(s, e) = \sum_{s', e'} P(s, e \mid s', e', a_{t-1}) b_{t-1}(s', e').$$

Lecture : « à partir de ce que nous croyions hier et de l'action réalisée, quels états sont plausibles maintenant ? »

2. Corriger avec l'observation reçue :

$$b_t(s, e) = \eta P(o_t | s, e, a_{t-1}) \tilde{b}_t(s, e),$$

où η normalise la somme des probabilités à 1.

Dans Step 2

La dynamique d'état était simplifiée : les deux états latents étaient tirés au début d'un épisode puis conservés. La fonction `joint_posterior` réalisait surtout l'étape de correction bayésienne à partir de l'historique observable. Un projet réel devrait souvent ajouter la prédiction temporelle : dérive du logiciel, changement de configuration, vieillissement des données, correction d'un défaut ou modification du banc de test.

G.4 Comment extraire $(\hat{s}, \hat{e})$ de la croyance ?

L'estimation ponctuelle peut être produite automatiquement à partir de b_t .

Maximum a posteriori :

$$(\hat{s}_t, \hat{e}_t)_{MAP} = \arg \max_{s, e} b_t(s, e).$$

Espérances, si les états sont numériques :

$$\hat{s}_t = \mathbb{E}[S_t | h_t], \quad \hat{e}_t = \mathbb{E}[E_t | h_t].$$

Marginales interprétables :

$$P(S_t = \text{mauvais} | h_t) = \sum_e b_t(\text{mauvais}, e),$$

$$P(E_t = \text{dégradée} | h_t) = \sum_s b_t(s, \text{dégradée}).$$

Dans un tableau de bord, on peut afficher ces résumés. Pour choisir une nouvelle preuve, il est préférable de conserver la distribution jointe complète ou une approximation qui permette de reconstituer ses incertitudes.

G.5 Quel estimateur utiliser ?

Il n'existe pas un estimateur universel. Le choix dépend du type d'état, de la dynamique et du degré de non-linéarité.

Situation	Famille d'estimateurs	Exemple QA / ingénierie
Quelques états discrets	Filtre bayésien exact, HMM, réseau bayésien dynamique	Bon/mauvais système, preuve fiable/dégradée, quelques régimes d'environnement.
État continu presque linéaire et bruit proche d'une loi gaussienne	Filtre de Kalman	Estimation d'une dérive progressive de latence ou de consommation mémoire.
Dynamique non linéaire	Kalman étendu ou unscented	Risque dépendant de charge, saturation, cache et file d'attente.
Distribution multimodale ou fortement non gaussienne	Filtre particulaire / Sequential Monte Carlo	Deux causes concurrentes restent plausibles : défaut produit ou banc de test dégradé.
Historique complexe et volumineux	Modèle d'état neuronal, Transformer temporel, inférence variationnelle	Fusion de tests, traces OTEL, logs, changements et incidents.

La méthode la plus sophistiquée n'est pas toujours la meilleure. Un petit réseau bayésien explicite peut être supérieur à un modèle profond si les données sont rares, les contraintes fortes et l'audit obligatoire.

G.6 Peut-on apprendre la croyance avec du machine learning ?

Oui. On peut apprendre un estimateur :

$$q_\phi(s, e | h_t) \approx P(S_t = s, E_t = e | h_t).$$

Le modèle reçoit l'historique et produit une distribution approximative. Cette approche est parfois appelée **inférence amortie** : le coût de l'inférence est déplacé vers l'entraînement, puis chaque nouvelle estimation devient rapide.

Les entrées possibles sont :

- résultats de tests et oracles ;
- traces, logs et métriques OTEL ;
- version, configuration, environnement et données ;
- défauts confirmés et incidents ultérieurs ;

— changements, déploiements, rollback et interventions humaines.

La sortie devrait idéalement conserver une incertitude : probabilités, quantiles, ensemble de particules, intervalle ou ensemble conforme. Un réseau qui produit seulement « risque = 0.72 » peut être utile, mais il ne fournit pas nécessairement la distribution requise pour calculer la valeur d’une nouvelle preuve.

G.7 Et avec un apprentissage non supervisé ?

Un apprentissage non supervisé peut découvrir des groupes ou régimes latents :

$$z_t = f_\theta(h_t), \quad z_t \in \{z_1, z_2, z_3, \dots\}.$$

Par exemple, il peut séparer statistiquement :

- un régime stable ;
- une dérive de performance ;
- une instrumentation incomplète ;
- une famille d’incidents intermittents.

Mais le modèle ne sait pas, par lui-même, que z_2 signifie « risque système élevé » et z_3 « qualité de preuve dégradée ». Cette sémantique doit être établie par :

- expertise métier et technique ;
- incidents connus ou labels rétrospectifs ;
- comparaison entre environnements ;
- interventions contrôlées ;
- sondes indépendantes et audits de traces ;
- validation de la stabilité du régime dans le temps.

Limite fondamentale

Un latent statistique n’est pas automatiquement un état métier. Le renommer « risque » ne le rend ni causal, ni identifiable, ni gouvernable. Il faut prouver que les variations de z correspondent à des mécanismes et conséquences utiles à la décision.

G.8 L’identifiabilité ne disparaît pas avec un modèle puissant

Supposons que deux causes produisent exactement la même loi observable :

$$P(o \mid S = \text{mauvais}, E = \text{bonne}) = P(o \mid S = \text{bon}, E = \text{dégradée}).$$

Aucun algorithme utilisant uniquement ces observations ne peut séparer les deux causes. Il peut prédire correctement la probabilité d’un FAIL, mais pas identifier pourquoi il survient.

Il faut alors modifier l’espace des observations ou intervenir :

- valider l’environnement avec un canal indépendant ;
- comparer deux bancs ou deux configurations ;
- injecter une panne ou un changement connu ;
- ajouter un oracle ou une sonde OTEL ;
- réaliser une revue humaine ciblée.

Cette distinction est cruciale :

$$\text{bonne prédiction} \not\Rightarrow \text{bonne identification du mécanisme.}$$

G.9 De la croyance à l’action : le rôle de la recherche opérationnelle

La recherche opérationnelle (RO) ne cherche pas d’abord à mieux estimer l’état. Elle cherche à **allouer des ressources et choisir des actions** sous contraintes.

À partir de b_t , elle peut résoudre :

$$a_t^* = \arg \min_a [C(a) + \mathbb{E}_{o|b_t, a} V(b_t^{a, o})].$$

Lecture : « choisir l’action dont le coût immédiat, ajouté au meilleur risque futur attendu après ses résultats possibles, est minimal ».

Dans un projet, les actions peuvent être :

- exécuter un test fonctionnel ou de performance ;
- valider l’environnement ou l’instrumentation ;
- demander une revue de sécurité ;
- déployer un canary, activer un contrôle compensatoire ou préparer un rollback ;
- arrêter et décider ;

— escalader vers une autorité disposant d’une compétence ou d’une légitimité supplémentaire.

Un POMDP appartient déjà à cette famille : lorsque les transitions, observations, pertes et contraintes sont connues, on peut calculer ou approximer une politique $\pi(b)$ par programmation dynamique, planification ou simulation.

G.10 Exemple complet : quelle preuve choisir maintenant ?

Supposons :

$$P(S = \text{mauvais}) = 0.18, \quad P(E = \text{dégradée}) = 0.35.$$

Action candidate	Coût	Information ou protection attendue
Test fonctionnel supplémentaire	1.0	Informe sur le système, mais son interprétation dépend de la qualité de preuve.
Validation d’environnement	0.5	Informe directement sur E et peut rendre les résultats fonctionnels interprétables.
Revue de sécurité	3.0	Recherche un risque rare mais très fortement pénalisé.
Arrêt et décision	0	Évite tout coût nouveau, mais conserve l’incertitude actuelle.

Une optimisation explicite peut conclure que la validation d’environnement doit précéder le test fonctionnel : apprendre sur E augmente la valeur des observations déjà obtenues et de celles qui suivront. La meilleure action n’est donc pas forcément celle qui a la plus grande probabilité brute de trouver un défaut.

G.11 Quand l’apprentissage par renforcement devient-il utile ?

L’apprentissage par renforcement (RL) devient pertinent lorsque le modèle complet n’est pas connu ou lorsque la séquence des actions produit des conséquences difficiles à programmer à la main.

Il peut apprendre une fonction de valeur :

$$Q(b, a) = \text{perte future attendue après l’action } a \text{ dans la croyance } b,$$

puis choisir :

$$\pi(b) = \arg \min_a Q(b, a).$$

Il existe plusieurs niveaux d’usage :

1. **Apprendre seulement le modèle** : estimer $P(o \mid s, e, a)$ ou les transitions, puis conserver une planification explicite.
2. **RL model-based** : apprendre un modèle du monde et planifier dedans.
3. **RL model-free** : apprendre directement Q ou la politique sans modèle causal explicite complet.
4. **Offline RL** : apprendre uniquement sur des historiques existants, sans exploration en production.

Pour une Test Authority, les deux premiers niveaux sont souvent les plus auditables. Le dernier est le plus risqué : une politique peut exploiter des corrélations ou des failles de récompense sans comprendre le mécanisme.

G.12 Recherche opérationnelle, POMDP, bandit ou RL ?

Approche	Quand elle suffit	Question type
Optimisation déterministe / programmation mathématique	Coûts et contraintes connus, peu d’incertitude sur l’état	Comment planifier une campagne sous budget et disponibilité des bancs ?
Analyse de décision / Bayesian experimental design	Une ou quelques acquisitions, modèle probabiliste explicite	Quel test a la plus grande valeur d’information nette ?
Bandit contextuel	Chaque choix donne un retour rapide et influence peu le futur	Quel test choisir maintenant pour ce contexte ?
POMDP planifié	L’état est caché, l’ordre des preuves et l’arrêt comptent, modèle disponible	Quelle séquence de preuves minimise le risque avant release ?
RL / POMDP appris	Modèle incomplet, nombreuses trajectoires historiques ou simulateur crédible	Quelle politique séquentielle apprendre à partir de l’expérience ?

Le RL n’est donc pas un signe automatique de maturité. Un bandit ou une optimisation à un pas peut être plus correct, plus explicable et plus facile à valider lorsque les effets futurs sont faibles.

G.13 Step 2 était-il du reinforcement learning ?

Non. Step 2 combinait :

— un modèle synthétique fixé ;

- une mise à jour bayésienne explicite ;
- une sélection selon une valeur décisionnelle à un pas ;
- un horizon fixe et des coûts unitaires égaux ;
- aucune amélioration de politique entre les épisodes ;
- aucun apprentissage à partir des résultats des épisodes précédents.

Il s'agissait donc principalement d'une **analyse décisionnelle model-based avec optimisation locale**, pas d'un entraînement RL.

Ce constat ouvre une suite logique, mais ne prouve pas qu'un RL ferait mieux. Step 2 a surtout isolé le problème à résoudre : une meilleure croyance reste souvent du même côté des seuils. Il faut donc étudier le couplage :

croyance $\rightarrow$ règle de décision $\rightarrow$ action $\rightarrow$ conséquence.

G.14 Pourquoi la RO ou le RL peuvent répondre au résultat de Step 2

Step 2 a observé que 1054 des 1119 améliorations de Brier ne changeaient pas l'action terminale. Une optimisation séquentielle plus riche pourrait explorer :

- l'arrêt adaptatif plutôt qu'un horizon imposé ;
- des coûts non unitaires et des ressources contraintes ;
- une action ESCALATE ;
- des contrôles compensatoires pour CONDITIONAL GO ;
- des décisions dépendant aussi de $P(E = \text{dégradée})$;
- la valeur future d'une preuve, pas seulement son effet à un pas ;
- une politique différente selon le prior, le contexte et la proximité des seuils.

Mais cette nouvelle politique devrait être testée sur un nouveau protocole et de nouvelles données. Elle ne doit pas être ajustée jusqu'à réussir sur les seeds déjà utilisées comme preuve confirmatoire.

G.15 La récompense ne doit pas remplacer la gouvernance

Un agent RL optimise ce qu'on lui donne, pas ce que l'organisation voulait implicitement dire. Une récompense mal formulée peut provoquer :

- la sélection de tests faciles à faire passer ;
- l'évitement d'investigations coûteuses mais critiques ;
- une inflation des CONDITIONAL GO ;
- l'amélioration d'une métrique intermédiaire sans réduction d'incident ;
- l'exploitation d'un défaut du simulateur ou du processus de mesure.

Une formulation gouvernée ressemble plutôt à :

$$\min_{\pi} \mathbb{E}_{\pi} [L_{\text{décision}} + C_{\text{preuves}} + C_{\text{retard}}]$$

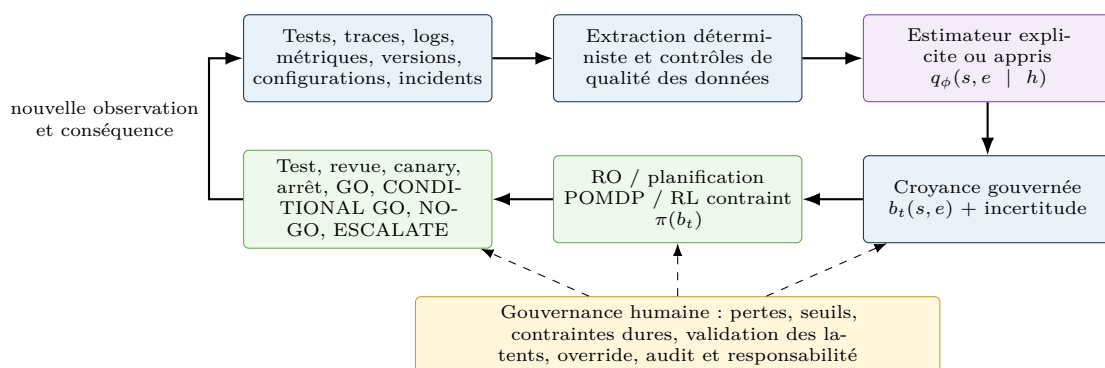
sous contraintes :

$$P_{\pi}(\text{unsafe GO}) \leq \varepsilon, \quad C_{\text{preuves}} \leq B,$$

preuves réglementaires exécutées, ESCALATE si autorité ou information insuffisante.

Ces contraintes ne sont pas de simples pénalités que l'agent peut échanger contre un gain ailleurs. Les exigences légales, de sécurité ou d'autorité peuvent être des **contraintes dures**.

G.16 Architecture industrielle hybride proposée



Cette architecture sépare les responsabilités :

1. les données brutes sont contrôlées avant tout apprentissage ;

2. le ML peut aider à estimer sans posséder l'autorité de décision ;
3. la croyance conserve l'incertitude et la provenance ;
4. la RO ou la politique choisit parmi des actions autorisées ;
5. les contraintes et décisions irréversibles restent gouvernées humainement.

G.17 Ce que cela implique pour le processus, les outils et les humains

Processus.

- enregistrer les hypothèses, priors, versions et changements de croyance ;
- relier chaque observation à son environnement, son oracle et sa provenance ;
- obtenir une vérité rétrospective lorsqu'elle devient disponible ;
- séparer entraînement, calibration, validation et confirmation ;
- surveiller dérive, changement de population et changement de politique ;
- autoriser l'abstention et l'escalade.

Outils.

- journal d'événements et evidence store ;
- registre de caractéristiques et de modèles ;
- simulateur ou jumeau contrôlé pour tester des politiques sans risque ;
- replay déterministe des décisions ;
- registre des seuils, fonctions de perte et contraintes ;
- tableaux de calibration, dérive et impact décisionnel.

Humains.

- le QA qualifie les oracles et limites de chaque observation ;
- le Tech Lead valide les mécanismes, configurations et causes plausibles ;
- le statisticien ou data scientist vérifie calibration, biais et pouvoir ;
- le métier, la sécurité, le juridique et le projet définissent les pertes non compensatoires ;
- l'autorité de décision fixe les contraintes, accepte le risque et reste responsable de l'override.

G.18 Les nouvelles hypothèses de recherche à formuler

Les discussions précédentes ne constituent pas encore des résultats. Elles produisent des hypothèses testables pour une étape future :

1. **Estimation** : un estimateur joint $q(S, E | h)$ correctement calibré distingue mieux défaut produit et preuve dégradée qu'un score de risque unique.
2. **Résumé** : une politique utilisant la croyance complète obtient de meilleures décisions qu'une politique utilisant seulement $(\hat{s}, \hat{e})$ à coût égal.
3. **Sémantique latente** : des régimes non supervisés ne deviennent utiles à la décision qu'après alignement métier et validation interventionnelle.
4. **Optimisation** : l'arrêt adaptatif et les coûts non unitaires convertissent davantage d'améliorations probabilistes en améliorations de décision.
5. **Apprentissage** : un modèle appris et une décision explicite surpassent une politique entièrement model-free en auditabilité, stabilité hors distribution et sécurité, sans perte majeure d'utilité.
6. **Gouvernance** : les contraintes dures et l'action ESCALATE réduisent les décisions dangereuses sans transformer systématiquement l'incertitude en NO-GO.

Chacune doit être préenregistrée avec sa population, ses données, sa métrique, ses baselines, ses conditions de réfutation et de nouvelles unités confirmatoires.

G.19 Mini-révision

1. Quelle différence sépare $b(s, e)$ de $(\hat{s}, \hat{e})$?
2. Deux croyances ayant le même MAP peuvent-elles justifier deux tests différents ? Pourquoi ?
3. Que réalisent les deux étapes « prédire » et « corriger » d'un filtre bayésien ?
4. Pourquoi un cluster non supervisé ne peut-il pas être automatiquement nommé « preuve dégradée » ?
5. Dans quel cas un bandit contextuel suffit-il, et dans quel cas faut-il un POMDP ?
6. Step 2 a-t-il entraîné une politique par reinforcement learning ?
7. Quel avantage présente « ML pour estimer + RO pour décider » ?
8. Pourquoi une contrainte de sécurité ne devrait-elle pas toujours être une simple pénalité dans la récompense ?
9. Quelles données rétrospectives sont nécessaires pour vérifier que la croyance et la politique étaient bonnes ?
10. Propose un exemple où ESCALATE est rationnellement préférable à un test supplémentaire.

G.20 Références méthodologiques

- [1] R. E. Kalman, « A New Approach to Linear Filtering and Prediction Problems », *Journal of Basic Engineering*, vol. 82, 1960.
- [2] L. R. Rabiner, « A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition », *Proceedings of the IEEE*, vol. 77, no 2, 1989.
- [3] N. J. Gordon, D. J. Salmond et A. F. M. Smith, « Novel Approach to Nonlinear/Non-Gaussian Bayesian State Estimation », *IEE Proceedings F*, vol. 140, no 2, 1993.
- [4] M. L. Puterman, *Markov Decision Processes : Discrete Stochastic Dynamic Programming*, Wiley, 1994.
- [5] L. P. Kaelbling, M. L. Littman et A. R. Cassandra, « Planning and Acting in Partially Observable Stochastic Domains », *Artificial Intelligence*, vol. 101, 1998.
- [6] R. S. Sutton et A. G. Barto, *Reinforcement Learning : An Introduction*, 2e édition, MIT Press, 2018.
- [7] E. Altman, *Constrained Markov Decision Processes*, Chapman & Hall/CRC, 1999.
- [8] Stanford AA228/CS238, *Decision Making under Uncertainty*, supports de cours et notes associées.
- [9] Dépôt ED-POMDP, modules de référence :
 - `benchmark/runtime/decision.py`;
 - `benchmark/runtime/ed_pomdp_policy.py`;
 - `benchmark/runtime/simulator.py`;
 - `benchmark/experiment/harness.py`.